

Machine Learning

Regression

IEEE SSCS Alexandria

by Marwan Abbas

Session Roadmap

01 Finding Minima with Derivatives

02 Newton's Method
From Idea to Algorithm

03 Linear Function & the Role of Bias/Weights

04 Loss Functions Making Error Logical

05 Gradient Descent

06 Normal Equation

07 Polynomial Regression

Notes



The Central Question

Given a function, how do we find its minimum?

Example: $f(x) = 3x^2 + 1$

$$\partial f(x) / \partial x = 6x$$

Set derivative = 0:

$$6x = 0 \rightarrow x = 0$$

$$\text{Substitute: } 3(0)^2 + 1 = 1$$

$\therefore$ Min value at point (0, 1)

↓ *Global minimum found using calculus*

But calculus only works for SIMPLE functions. Real ML cost functions are far more complex...

Newton's Method — The Idea

We want to find where $f'(x) = 0$ (the minimum of a function)

$$x_{i+1} = x_i - f'(x_i) / f''(x_i)$$

How it works:

- Start at any random point x_i
- Draw the tangent line at that point (slope = $f'(x_i)$)
- Find where that tangent hits the x-axis → that's x_{i+1}
- Repeat until convergence (x barely changes)

Example: $f(x) = x^4 - 3x^3 + 2$

$f'(x) = 4x^3 - 9x^2$

$f''(x) = 12x^2 - 18x$

Start: $x = 5$

①	3.690476
②	2.881513
③	2.442450
④	2.276199
⑤	2.250589
⑥	2.250000

Newton's Method — Why Not in ML?

Pros

- Converges quickly for well-behaved functions
- Mathematically elegant
- Precise when it works

Cons

- Unstable & slow for complex functions
- Not efficient for high-dimensional ML
- Can oscillate around minimum
- Requires computing second derivatives
- Does NOT scale to millions of parameters

⇒ **We need a better approach for Machine Learning.
Enter: Gradient Descent!**

Notes



Linear Function — Building Blocks

bias (intercept)
→ shifts line up/down

$$y = \alpha + \beta \cdot x$$

weight (slope)
→ direction & steepness

Case 1: $\alpha = 0$, vary β

All lines pass through origin. β controls steepness: +5, +4, +3, +2, +1, -1, -2 ...

Case 2: β fixed, vary α

Parallel lines shifted up ($\alpha=2,1,0$) or down ($\alpha=-1,-2$). Same slope, different intercept.

Together

α and β together can represent ANY line in 2D space.

Notes



The Problem — How Does the Computer Learn α & β ?

Real-world example: Predicting car price from year

Year	Price (k)
2020	400
2021	500
2022	550
2023	600
2024	750
2025	780
2026	800

Learning Algorithm Steps:

- ① Give α and β random values
- ② Calculate predicted $\hat{y}$ using $h(x) = \alpha + \beta x$
- ③ Calculate error between $\hat{y}$ and actual y
- ④ Update α & β to reduce error
- ⑤ Repeat until error is minimized

Loss Functions — Making Error Logical

We need a number that tells us: "How wrong is our model?"

Why not just sum the errors? $\text{error} = h(x) - y$

Problem: positive & negative errors cancel each other out!

Solution: Square them → Mean Squared Error (MSE)

$$J(\theta) = (1/2m) \sum (h(x) - y)^2$$

$J(\theta) \uparrow$ means error is HIGH → bad model

$J(\theta) \downarrow$ means error is LOW → good model

Goal: Find θ that makes $J(\theta)$ as small as possible

Mean Squared Error (MSE)

MSE Formula:

$$J(\theta) = (1/2m) \sum_i (h\theta(x) - y)^2$$

Pros

- Penalizes larger errors more heavily (due to squaring)
- Differentiable — suitable for optimization

Numerical Example ($\alpha=1$, $\beta=2$):

$h-y$	$(h-y)^2$
3641	13,256,881
3543	12,552,849
3495	12,215,025
3299	10,883,401

Cons

- Sensitive to outliers (large errors are amplified)

Other Loss Functions — MAE & Huber

Mean Absolute Error (MAE)

$$J(\theta) = (1/2m) \sum |h(x) - y|$$

Less sensitive to outliers | Simpler interpretation (avg absolute error)

Non-differentiable at 0 → optimization harder | May not penalize large errors as effectively

Huber Loss

$$\frac{1}{2}(h-y)^2 \text{ if } |h-y| \leq \delta$$

$$\delta|h-y| - \delta^2/2 \text{ if } |h-y| > \delta$$

Best of both: MSE for small errors, MAE for large errors | Robust to outliers

Requires tuning of δ parameter

Notes



Gradient Descent — Core Idea

Iterative optimization: adjust weights step-by-step to minimize $J(\theta)$

$$\theta_j := \theta_j - \mu \cdot \nabla J(\theta)$$

$\nabla J(\theta)$ = gradient of the cost function (direction of steepest ascent)

We subtract the gradient → we move opposite to steepest ascent → toward the minimum

For i in range(iterations):

→ Calculate $h(\theta)$ → Calculate $\nabla J(\theta)$ → Update weights θ

Gradient Descent — Computing the Gradient

For $h(x) = \alpha + \beta x$ and $J = (1/2m) \sum (\alpha + \beta x - y)^2$

Case 1: Gradient w.r.t. α (θ_0)

$$\partial J / \partial \alpha = (1/m) \sum (\alpha + \beta x - y)$$

Sum of all prediction errors

Case 2: Gradient w.r.t. β (θ_1)

$$\partial J / \partial \beta = (1/m) \sum (\alpha + \beta x - y) \cdot x$$

Sum of errors weighted by input x

$$\therefore \theta_j := \theta_j - \alpha \cdot (1/m) \sum (h(\theta) - y) \cdot x_j$$

Learning Rate (μ) — The Critical Hyperparameter

Controls how big each update step is in the direction opposite the gradient

Too Small ($\mu \rightarrow 0$)

Small steps $\rightarrow$ very stable but very slow convergence. Many iterations needed. May get stuck.

Just Right $\checkmark$

Smooth convergence to the minimum. The sweet spot that balances speed and stability.

Too Large ($\mu \gg 0$)

Large steps $\rightarrow$ overshoots the minimum. Loss oscillates or diverges. Unstable training.

There is no universal best μ — it must be tuned for each problem. Try values like 0.01, 0.001, 0.1 and observe the loss curve.

Notes



Normal Equation — Analytical Solution

Instead of iterating, can we solve for optimal θ directly? YES!

Derivation Summary

① Write $J(\theta) = \|X\theta - y\|^2 = (X\theta - y)^T(X\theta - y) = \theta^T X^T X \theta - 2y^T X \theta + y^T y$

② Take derivative w.r.t. $\theta \rightarrow 2X^T X \theta - 2X^T y$

③ Set derivative = 0 $\rightarrow X^T X \theta = X^T y$

④ Solve: $\theta = (X^T X)^{-1} X^T y$

$$\theta = (X^T X)^{-1} X^T y$$

← Optimal parameters (closed-form, no iteration needed)

Normal Equation vs Gradient Descent

	Normal Equation	Gradient Descent
Iteration needed?	No	Yes
Learning rate?	Not required	Must be tuned
Complexity	$O(n^3)$ — matrix inverse	$O(kn)$ per iteration
Best for	Small datasets / few features	Large datasets / many features
Practical limit	~10,000 features max	Scales to millions

Rule of thumb: If $n < 10,000$ features → Normal Equation. Otherwise → Gradient Descent.

Notes



Linear Regression — Full Pipeline

1

Collect Data

Gather features X and labels y

2

Choose Model

$$h(x) = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n = \theta^T X$$

3

Define Loss

$$J(\theta) = (1/2m) \sum (h\theta(x) - y)^2 \quad [\text{MSE}]$$

4

Optimize

Gradient Descent or Normal Equation to minimize $J(\theta)$

5

Predict

For new x : $\hat{y} = \theta^T x$

Data → Model → Loss → Optimize → Predict → Evaluate → Repeat

Key Takeaways

Derivatives find minima analytically — but only for simple functions

Newton's Method improves on calculus but is unstable for ML scale

Linear functions $y = \alpha + \beta x$: α shifts, β rotates — together form any line

Loss functions quantify error: MSE (squares errors), MAE (absolute), Huber (hybrid)

Gradient Descent iteratively minimizes loss using the update rule $\theta := \theta - \mu \nabla J(\theta)$

Normal Equation gives closed-form optimal $\theta = (X^T X)^{-1} X^T y$ — best for small n

Learning rate μ is critical: too small = slow, too large = diverges

Notes

